ЛАБОРАТОРНА РОБОТА № 2

Тема:
Unit-тестування асинхронного коду з Jest

Мета:
Метою лабораторної роботи є набуття практичних навичок написання автоматизованих тестів для асинхронного TypeScript-коду з використанням фреймворку Jest, зокрема: • налаштування Jest з TypeScript (ts-jest); • написання unit-тестів для асинхронних функцій; • використання матчерів Jest (toBe, toEqual, toThrow, toHaveBeenCalledTimes тощо); • тестування Promise-based коду через async/await та .resolves/.rejects; • використання lifecycle hooks (beforeEach, afterEach) та fake timers (jest.useFakeTimers); • вимірювання покриття коду (code coverage); • застосування generics у TypeScript для типізації тестів.

Хід роботи:

Завдання №3.1-3.2

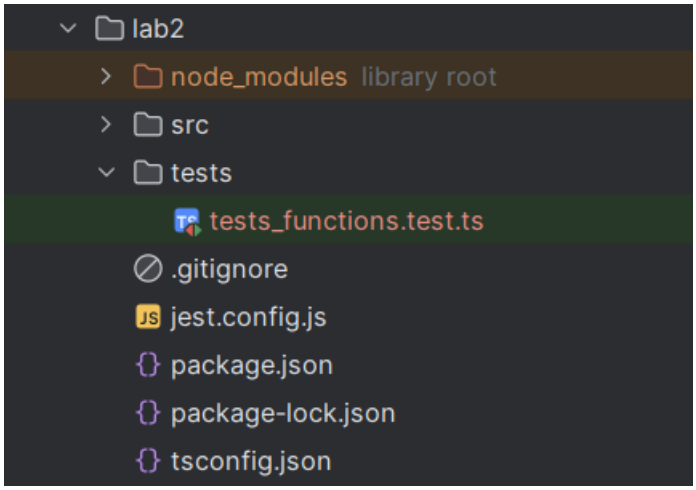

Рис.1 Структура проекту

Завдання №3.3

Написати тести для функції fetchUserProfiles з Лабораторної роботи №1. Сигнатура: fetchUserProfiles(userIds: string[]): Promise Вимоги: • Мінімальна кількість тестів: 3.

Лістинг програми тести до функції fetchUserProfiles():

```
describe('fetchUserProfiles()', () => {
    it('має повернути порожній масив якщо передано порожній ID', async () => {
        const input: string[] = [];

        const result = await fetchUserProfiles(input);
        expect(result).toEqual([]);
    });
```

```
    it('має повернути коректно сформовані профілі для декількох ID', async ()
=> {
        const id = ['1', '2'];

        const result = await fetchUserProfiles(id);
        expect(result).toHaveLength(2);

        expect(result[0]).toEqual({
            id: '1',
            name: 'User 1',
            email: '1@example.com',
        });
    });

    it('Кожен профіль має містити коректний email відповідно до ID', async ()
=> {
        const id = ['test-user'];
        const [user] = await fetchUserProfiles(id);

        expect(user.email).toBe('test-user@example.com');
    });
});
```

```
PASS  tests/tests_functions.test.ts
  fetchUserProfiles()
    √ має повернути порожній масив якщо передано порожній ID (2 ms)
    √ має повернути коректно сформовані профілі для декількох ID (132 ms)
    √ Кожен профіль має містити коректний email відповідно до ID (109 ms)
```

Рис.2 Результат виконання тестів

Завдання №3.4

Тести для retryOperation Написати тести для функції retryOperation з Лабораторної роботи №1. Сигнатура: retryOperation(operation: () => Promise, maxRetries: number): Promise Вимоги: • Мінімальна кількість тестів: 3.

Лістинг програми тести до retryOperation:

```
describe('retryOperation()', () => {
    it('Вдала операція з першої спроби', async () => {
        let attempts = 0;

        const mockOp = async () => {
            attempts++;
            if (attempts < 2) throw new Error('Помилка');
            return "Успіх";
        };

        const result = await retryOperation(mockOp, 3);
        expect(result).toBe("Успіх");
        expect(attempts).toBe(2);
    });

    it('Помилка якщо всі спроби вичерпано', async () => {
        const failingOp = async () => { throw new Error('Нічого не вийде');};
        await expect(retryOperation(failingOp, 2))
            .rejects
            .toThrow("Всі спроби вичерпано");
    });

    it('має повернути успіх з 3 спроби', async () => {
```

```
        let attempts = 0;

        const mockOp = async () => {
            attempts++;
            if(attempts < 3) throw new Error('тимчасова помилка');
            return 'Успіх';
        };
        const result = await retryOperation(mockOp, 5);

        expect(result).toBe('Успіх');
        expect(attempts).toBe(3);
    });
});
    return Promise.all(promises);
}
```

```
retryOperation()
  √ Вдала операція з першої спроби (134 ms)
  √ Помилка якщо всі спроби вичерпано (119 ms)
  √ має повернути успіх з 3 спроби (233 ms)
```

Рис.3 Результат виконання тестів

Завдання №3.5

Тести для processInBatches Написати тести для функції processInBatches з Лабораторної роботи №1. Сигнатура: processInBatches(items: T[], batchSize: number, processor: (batch: T[]) => Promise): Promise Вимоги: • Мінімальна кількість тестів: 3.

Лістинг програми тести до processInBatches:

```
describe('processInBatches()', () => {
   it('Повинен обробляти масив чисел частинами', async () => {
       const input = [1, 2, 3, 4];
       const batchSize = 2;

       const mockProcessor =
           async (batch: number[])=> batch.map(n => n * 10);

       const result = await processInBatches(input, batchSize, mockProcessor);

       expect(result). toEqual([10, 20, 30, 40]);
   });

   it('Повинен обробляти масив рядків' , async () => {
      const  input = ["a", "b"];

      const mockProcessor =
          async (batch: string[]) => batch.map(s => s.toUpperCase());

      const result = await processInBatches(input, 1, mockProcessor);

      expect(result).toEqual(["A", "B"]);
   });

   it('має трансформувати типи даних з number до string', async () => {
      const  input = [1, 2, 3];

      const mockProcessor = async (batch: number[])=>
          batch.map(n => `ID: ${n}`);

      const result = await processInBatches(input, 2, mockProcessor);
```

```
        expect(result).toEqual(["ID: 1", "ID: 2", "ID: 3"]);
    });
});
```

```
processInBatches()
  √ Повинен обробляти масив чисел частинами (2 ms)
  √ Повинен обробляти масив рядків (3 ms)
  √ має трансформувати типи даних з number до string (3 ms)
```

Рис.4 Результат виконання тестів

Завдання №3.6

Тести для raceWithTimeout Написати тести для функції raceWithTimeout з Лабораторної роботи №1. Сигнатура: raceWithTimeout(promise: Promise, timeoutMs: number): Promise Вимоги: • Мінімальна кількість тестів: 2.

Лістинг програми тести до raceWithTimeout:

```
describe('raceWithTimeout()', () => {
    it('результат якщо операція встигла', async () => {
        const fastPromise = new Promise(resolve =>
            setTimeout(() => resolve('Встиг'), 50));

        const result = await raceWithTimeout(fastPromise, 100);

        expect(result).toBe('Встиг');
    });

    it('Викине помилку якщо час вийшов', async () => {
        const slowPromise = new Promise(resolve =>
        setTimeout(() => resolve('Не встиг'), 200));

        await expect(raceWithTimeout(slowPromise, 100))
            .rejects
            .toThrow('Operation timed out after 100ms');
    });
});
```

```
raceWithTimeout()
  √ результат якщо операція встигла (52 ms)
  √ Викине помилку якщо час вийшов (111 ms)
```

Рис.5 Результат виконання тестів

Висновок: В даній лабораторній я набув практичних навичок написання автоматизованих тестів для асинхронного TypeScript-коду з використанням фреймворку Jest зокрема: • налаштування Jest з TypeScript (ts-jest); • написання unit-тестів для асинхронних функцій; • використання матчерів Jest (toBe, toEqual, toThrow, toHaveBeenCalledTimes тощо); • тестування Promise-based коду через async/await та .resolves/.rejects; • використання lifecycle hooks (beforeEach, afterEach) та fake timers (jest.useFakeTimers); • вимірювання покриття коду (code coverage); • застосування generics у TypeScript для типізації тестів.